

UAQE is an autonomous, capability-driven AI model optimization and orchestration engine that: (1) accepts supported AI models and datasets via drag-and-drop ingestion, (2) inspects metadata, shapes, and task contracts, (3) establishes empirical FP32 baseline metrics, (4) autonomously generates candidate optimization strategies, (5) executes calibration-guided quantization (INT8), sensitivity-aware structured pruning, and lossless weight compression, (6) benchmarks pure inference latency, memory (RSS), and accuracy, (7) enforces strict percentage-point safety policies, (8) selects the verified Pareto winner, and (9) packages a cryptographically hashed, deployable edge bundle.

THE FIVE CANONICAL CONCEPTUAL LAYERS OF UAQE				
1. UNDERSTAND	2. TRANSFORM	3. EXECUTE	4. MEASURE	5. DECIDE
Inspect model graph, dtypes, task signature, and dataset distribution.	Apply INT8 QDQ / TFLite quantization, pruning, and Huffman/RLE compression.	Invoke target runtime (ONNXRuntime / TFLite Interpreter) on real input batches.	Profile pure inference latency, throughput, RSS memory, and accuracy loss.	Gating policy: classify candidates into EXCELLENT, ACCEPTABLE, or CRITICAL.
FP32 TRAINED MODEL	→ CALIBRATION & QUANTIZATION	→ INT8 SERIALIZED ARTIFACT	→	EDGE RUNTIME (SIMD/NEON)

PHASE 01

NEURAL NETWORK FUNDAMENTALS & EDGE COMPUTE PRIMITIVES

Neuron Mathematical Formulation: An artificial neuron evaluates $y = f(\mathbf{w} \cdot \mathbf{x} + b)$, where $\mathbf{w}$ is the weight vector, $\mathbf{x}$ is the input activation vector, b is the scalar bias, and $f(\cdot)$ is a non-linear activation function. **Weights:** Learnable kernel parameters that capture spatial/semantic correlation. **Bias:** Learnable offset ensuring activation independence from origin. **Layers:** Successive functional blocks (Dense/Linear, Convolutional 2D, Depthwise Separable, Pooling, Normalization). **Tensors & Shapes:** Multidimensional numerical arrays; e.g., PyTorch vision tensors use NCHW [Batch, Channels, Height, Width], whereas edge runtimes (TFLite/Edge TPU) prefer NHWC [Batch, Height, Width, Channels] for contiguous memory spatial locality. **Data Types (dtype):** Bit representation determining dynamic range and arithmetic hardware path. **Activations & Logits:** Unnormalized real outputs prior to normalization. **Softmax:** Normalizes logits into probabilities: $P(y) = \exp(z_y) / \sum \exp(z_j)$. **Convolution & Feature Maps:** Sliding 2D kernel computing cross-correlations; output channels represent learned feature filters. **MAC (Multiply-Accumulate):** The fundamental atomic operation of deep learning: $Acc = Acc + (\mathbf{w}_i \times \mathbf{x}_j)$. One MAC comprises 1 multiplication + 1 addition = 2 FLOPs. Edge optimization focuses directly on replacing 32-bit FP MACs with 8-bit INT MACs.

PHASE 02

MODEL STORAGE, SERIALIZATION FORMATS & GRAPH REPRESENTATION

Unified Definition: A deployable deep learning model strictly consists of: **Model = Architecture Graph + Parameter Weights + Serialization Metadata.**

- .pt / .pth (PyTorch):** Pickled Python bytecode storing state dictionaries. High flexibility during training, but dangerous for edge deployment due to arbitrary code execution risks and lack of native embedded C++ runtime support.
- SafeTensors:** Hugging Face zero-copy, memory-mapped tensor format. Safe against arbitrary code execution; contains no executable graph representation.
- ONNX (Open Neural Network Exchange):** Protocol Buffer graph serialization representing operators as standard schemas. Ideal for server/desktop cross-platform inference via ONNX Runtime (ORT); uses QDQ nodes (QuantizeLinear / DequantizeLinear) for simulated and true INT8 execution.
- TFLite (.tflite):** FlatBuffers-based binary format designed for microcontrollers, mobile, and edge Linux boards. Zero parsing overhead (memory-mappable directly from flash memory into RAM); stores per-channel quantization scale and zero-point vectors directly in operator headers.
- Bare-Metal Embedded Formats (.bin / .mem / .hex):** Raw contiguous byte streams or hex dumps utilized by FPGA block RAM, DSPs, and custom ASIC DMA controllers.
- Storage Representation vs. Runtime Representation:** On-disk compressed or pruned weights (e.g., zipped or sparse-indexed) occupy minimal flash storage, but must be decompressed or mapped into active DRAM tables upon invocation by runtime kernels.
- Graph Metadata:** Embedded input/output tensor names, expected shapes, quantization scale/zero-point arrays, and operator domain versions.

PHASE 03			
NUMERICAL PRECISION & QUANTIZATION PARADIGMS			
Precision	Bits / Bytes	Dynamic Range	Edge Hardware Suitability & UAQE Context
FP32 (Single)	32 b / 4 B	1.18e-38 to 3.40e+38	Baseline training precision; excessive memory bandwidth and energy on edge CPUs.
FP16 (Half)	16 b / 2 B	5.96e-8 to 65504	Reduces memory by 50%; supported on modern mobile GPUs and Apple Neural Engine.
INT8 (Integer)	8 b / 1 B	[-128, 127] or [0, 255]	UAQE Core Target: 4× memory footprint reduction, 2–4× compute throughput via SIMD.
INT4 (Sub-byte)	4 b / 0.5 B	[-8, 7] or [0, 15]	High compression for LLM weight-only quantization; severe accuracy loss on small CNNs.

Scale (S): Positive real floating-point scalar mapping continuous real numbers into discrete integer steps: $S = (max - min) / (q_{max} - q_{min})$. **Zero Point (Z):** Integer value corresponding exactly to real zero (0.0), guaranteeing that real zero (frequently padded in CNN convolutions) maps without error. **Symmetric vs. Asymmetric:** Symmetric sets $Z = 0$ (maps [-max, +max] to [-127, 127]), eliminating zero-point cross-term math in MAC loops. Asymmetric sets $Z \neq 0$, preserving high resolution for strictly non-negative activation distributions (e.g., ReLU outputs). **Per-Tensor vs. Per-Channel:** Per-tensor shares one (S, Z) across an entire layer. Per-channel assigns an individual scale S_c per output filter, preventing a single large outlier weight in one channel from compressing the numerical dynamic range of all other channels. **INT8 × INT8 → INT32 Accumulator:** Multiplying two 8-bit signed integers yields up to a 15-bit signed product ($127 \times 127 = 16,129$). Summing these across 512 or 1024 channels easily overflows an 8-bit or 16-bit register. Edge hardware strictly utilizes **32-bit integer accumulators**.

PHASE 04	QUANTIZATION MATHEMATICS, NUMERICAL FORMULATIONS & BIAS SCALING	
Symmetric Quantization: $q = \text{round}(x / S)$ Dequantization: $x \blacksquare = q \times S$	Asymmetric Quantization: $q = \text{clamp}(\text{round}(x / S) + Z, q_{\min}, q_{\max})$ Dequantization: $x \blacksquare = (q - Z) \times S$	Requantization Multiplier: $M = (S_x \times S_w) / S_y$ Fixed-Point Implementation: $M \approx M_0 \times 2^{-n}$

Mathematical Parameter Definitions:

- Scale Factor (S):** Determined by dynamic range clipping: $S = (x_{\max} - x_{\min}) / (q_{\max} - q_{\min})$.
- Zero Point (Z):** Integer alignment: $Z = \text{round}(-x_{\min} / S) + q_{\min}$. Guarantees exact representation of 0.0 to prevent zero-padding errors.
- Clipping, Saturation & Rounding:** Floating values outside $[x_{\min}, x_{\max}]$ are clamped to avoid integer wrap-around. Nearest integer rounding introduces round-off error $\epsilon \leq S/2$.
- Outlier Management:** Activations often follow heavy-tailed Gaussian distributions. A single extreme outlier will drastically inflate S, collapsing 99% of normal weights into 2 or 3 discrete integer levels. UAQE handles this through percentile calibration and per-channel weight scaling.
- Bias Quantization Math:** In a convolution layer, the accumulator computes: $Accumulator = \sum (x_q \times w_q)$. The effective scale of each product is $S_x \times S_w$. To add the bias directly to the accumulator without expensive runtime alignment, **the bias is quantized with scale $S_{bias} = S_x \times S_w$** and stored as a **32-bit integer (INT32)**.
- Requantization Down-Scaling:** The 32-bit accumulated sum must be converted back to an 8-bit integer activation y_q with scale S_y before passing to the next layer: $y_q = \text{round}(Accumulator \times M) + Z_y$. To avoid floating-point hardware on edge chips, M is factored into a 32-bit fixed-point integer multiplier $M_0 \in [0.5, 1.0) \times 2^{31}$ and a right-shift exponent n .

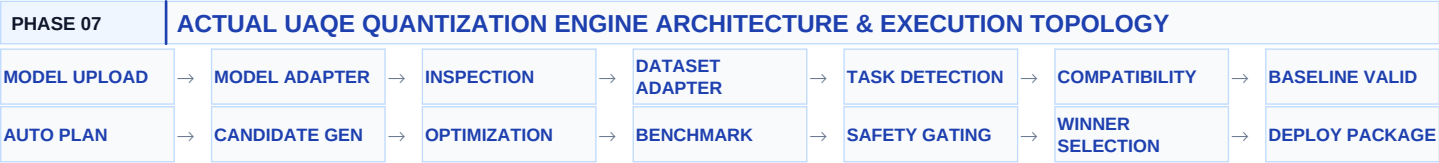
PHASE 05	CALIBRATION METHODOLOGIES, PTQ & QUANTIZATION-AWARE TRAINING (QAT)			
FP32 PRETRAINED MODEL	→	REPRESENTATIVE CALIBRATION DATA	→	OBSERVERS (MinMax / Histogram) → INT8 ARTIFACT EXPORT
FP32 TRAINED MODEL	→	INSERT FAKE-QUANT NODES	→	FINETUNE (STE BACKPROP) → INT8 FROZEN EXPORT

Calibration: The mathematical process of feeding representative unlabelled input data through the model to record activation distributions, enabling optimal selection of scale S and zero-point Z without modifying weights.

- Observers:** Profiling modules placed at tensor boundaries. *MinMaxObserver* tracks absolute bounds [min, max] (fast, but susceptible to outliers). *Histogram / Percentile Observer* clips the top 0.01% extreme outliers. *Entropy / KL-Divergence Observer* minimizes the Kullback-Leibler divergence between the FP32 distribution and the quantized INT8 distribution (widely adopted by TensorRT).
- Representative Dataset:** A curated subset of calibration samples (typically 100–500 images) matching the target deployment sensor domain.
- Weight Calibration:** Static and deterministic; computed directly from frozen weight tensors without requiring data batches.
- Activation Calibration:** Dynamic and data-dependent; must capture realistic input variance.
- Fake Quantization in QAT:** Nodes inserted during training that discretize activations and weights to simulate INT8 rounding error during the forward pass, while storing gradients and parameters in FP32: $x_{fake} = S \times (\text{clamp}(\text{round}(x/S) + Z, q_{\min}, q_{\max}) - Z)$.
- Straight-Through Estimator (STE):** Because the derivative of the step-rounding function is 0 everywhere (and undefined at boundaries), standard backpropagation cannot train quantized networks. The STE replaces the non-differentiable gradient with an identity pass-through: $\partial \text{round}(x) / \partial x \approx 1$ for $x \in [x_{\min}, x_{\max}]$, and 0 outside, allowing full gradient descent.

PHASE 06	PTQ VS QAT IN UAQE: PRODUCTION WORKFLOWS & PERFORMANCE DECOUPLING		
Evaluation Dimension	Post-Training Quantization (PTQ)	Quantization-Aware Training (QAT)	UAQE Architectural Role & Findings
Retraining Required?	No — zero parameter retraining	Yes — fine-tuning for several epochs	PTQ offers immediate candidate evaluation (seconds vs hours).
Compute / Time Cost	Very low (seconds on CPU)	High (requires GPU training loop)	UAQE autonomous search runs PTQ exploration passes first.
Accuracy Preservation	Good for large models; drops on small CNNs	Superior — recovers >99% of FP32 baseline	QAT used when PTQ exceeds critical loss threshold (>4.0 pp).
Verified Project Usage	ResNet-50 + CIFAR-10: ONNX QDQ static PTQ	MobileNet: True INT8 TFLite (QAT history)	Verified: ResNet ONNX QDQ achieves robust PTQ accuracy.

[CRITICAL DISTINCTION: ACCURACY RECOVERY VS. RUNTIME SPEEDUP] QAT DOES NOT AUTOMATICALLY GUARANTEE FASTER INFERENCE. QAT only alters weight distributions to tolerate precision loss; it produces the exact same INT8 tensor format as PTQ. Actual runtime inference speedup depends *strictly* on execution kernels (e.g. XNNPACK, AVX-512 VNNI, ARM NEON), memory bandwidth, cache hit rates, and tensor layout transforms (NCHW ↔ NHWC). Quantization accuracy and runtime performance are completely decoupled engineering concerns.



- Architectural Subsystems & Core Engineering Principles:**
- **Model Adapters:** Abstract away framework-specific details (PyTorch, ONNX, TFLite), exposing a uniform graph interface for inspection, node traversal, and parameter extraction.
 - **Dataset Adapters:** Ingest heterogeneous inputs (ImageFolder directory structures, CIFAR-10 binary pickles, single images), enforcing strict input shape resolution, normalization, and tensor batching.
 - **Optimization Planner & Orchestrator:** Evaluates target hardware capabilities against model architecture, generating a deterministic matrix of candidate optimization strategies.
 - **Candidate Evaluator & Job Isolation:** Every candidate executes in a dedicated sandbox directory (output/jobs/<id>). Telemetry and metrics are logged independently, completely preventing state leakage across candidate runs.
 - **Baseline Validity Preflight:** Before any optimization begins, the unquantized FP32 model is evaluated on the validation dataset. If the baseline fails to produce valid outputs or accuracy, the entire pipeline immediately halts.
 - **Capability-Driven Architecture (Universal ≠ Every Architecture):** UAQE operates on verified operator capabilities. It supports verified CNN pipelines (MobileNet, ResNet-50) while cleanly detecting and rejecting unsupported architectures (e.g., Vision Transformers).



- Micro-Architectural Mechanics of INT8 Vector Execution:**
- **Convolution & MAC Loop:** An edge CPU processes convolution by unfolding input patches into contiguous memory (im2col) and executing General Matrix Multiply (GEMM). Four 8-bit integers are packed into a single 32-bit register and multiplied in parallel.
 - **SIMD Acceleration (ARM NEON & x86 VNNI):** Modern edge cores feature SIMD vector registers (128-bit NEON on Cortex-A76, 256-bit AVX2 / 512-bit AVX-512 on x86). Instructions such as ARM SDOT (Signed Dot Product) multiply four pairs of INT8 values and accumulate into a 32-bit vector lane in a single clock cycle.
 - **XNNPACK Operator Concept:** A highly tuned assembly micro-kernel library targeting ARM, x86, and WebAssembly. It mandates NHWC memory layout and packs weights into specialized cache-friendly blocks.
 - **Cache Hierarchy & Memory Bandwidth:** In FP32 inference, fetching 4-byte weights frequently saturates external DRAM bus bandwidth, causing CPU stalls. INT8 reduces memory traffic by 75%, allowing entire model layer weights to reside within fast L2/L3 cache.
 - **Cold-Start vs. Warm Inference:** Cold-start latency includes graph loading, memory allocation (allocate_tensors), kernel compilation, and instruction cache warming. Warm inference measures steady-state execution after memory buffers and caches are fully primed.

Phase 09	Pruning, Sparsity Paradigms & Lossless Artifact Compression		
Pruning Paradigm	Sparsity Pattern	Hardware Acceleration Feasibility	Storage vs. Runtime Impact
Unstructured Pruning	Fine-grained, random zero weights throughout tensor.	Poor — standard edge CPUs cannot skip random zeros; requires dense compute.	High storage compression (via gzip/RLE); 0% speedup on standard CPUs.
Structured Pruning	Coarse-grained; removes entire channels or filters.	Excellent — directly shrinks matrix dimensions; native speedup on any engine.	Reduces both storage size AND runtime latency proportionately.
Lossless Compression	Huffman / Deflate / Run-Length Encoding (RLE).	Applied during on-disk packaging; requires decompression pass before inference.	Reduces flash memory footprint; does NOT alter runtime model RAM.
[CARDINAL RULE OF COMPRESSION] Sparsity % ≠ Storage Reduction % ≠ Runtime Latency Speedup %. A model with 50% unstructured weight sparsity still requires 100% of dense floating-point or integer operations on standard ARM Cortex-A CPU cores. Furthermore, uncompressed zeros still consume 1 byte each in uncompressed flat file formats. Speedup is achieved only when structured channel pruning reduces matrix dimensions or when runtime kernels natively support sparse block indexing.			

PHASE 10

MODEL COMPRESSION & ARTIFACT ENGINEERING: STORAGE VS. RUNTIME REPRESENTATION

Artifact Engineering Principles & Packaging Lifecycle:

- **Deployable Artifact:** A standalone, self-contained deployment bundle containing the runtime model binary, cryptographic verification hashes, hardware metadata, and execution contracts.
- **Storage Representation vs. Runtime Representation (Verified UAQE Empirical Finding):** An optimized model packaged with sensitivity-aware pruning and lossless stream compression can occupy **1.39 MB** on disk. However, when loaded into the edge runtime (e.g. TFLite Interpreter), the flat binary tensor arrays expand into their full uncompressed representation of **1.86 MB** in RAM. Engineers must never confuse compressed distribution archive size with active memory footprint.
- **Reconstruction & Decoder Overhead:** Unpacking compressed archives introduces a small CPU and RAM overhead during cold boot. UAQE profiles this reconstruction latency to verify that edge startup limits are strictly respected.
- **Cryptographic Provenance (SHA-256):** Every raw model file, calibration dataset slice, intermediate candidate, and final deployment package is hashed with SHA-256. The resulting checksums are embedded in a tamper-proof manifest.json to guarantee auditability and supply chain integrity.

PHASE 11

BENCHMARKING & PERFORMANCE ENGINEERING: LATENCY, THROUGHPUT & TELEMETRY ISOLATION

Throughput (Frames Per Second): Throughput = 1000 / Latency _{ms}	Latency Speedup Percentage: Speedup % = ((Baseline Latency - Optimized Latency) / Baseline Latency) × 100
--	--

Rigorous Performance Measurement Methodology:

- **Pure Inference vs. End-to-End Latency:** Pure inference measures only model graph compute (interpreter.invoke()). End-to-end latency includes image decoding, resizing, RGB normalization, tensor copy, and argmax postprocessing. Both must be reported separately.
- **Statistical Rigor:** A single timing run is meaningless. UAQE discards initial warmup passes (cache priming), then samples 50–200 timed iterations to calculate **Mean, Median (P50), P95, and P99 tail latency**.
- **Memory Profiling:** Tracks OS Resident Set Size (RSS) and Peak Allocation using platform system calls, isolating memory consumption from garbage collection spikes.
- **Telemetry Contamination Avoidance:** Profiling hooks must not run inside the timing loop. Telemetry collection is isolated to dedicated sampling threads to prevent observer overhead.
- **Verified MobileNet XNNPACK Compatibility Finding:** INT8 is NOT automatically faster. If a runtime delegate encounters unsupported tensor layouts or missing kernel patterns, it falls back to slow reference kernels. In our verified testbed, the MobileNet INT8 model runs on the verified stable reference TFLite path rather than assuming XNNPACK acceleration.

PHASE 12

AUTONOMOUS OPTIMIZATION, CANDIDATE SEARCH & SAFETY POLICY GATING

Classification	Accuracy Loss Threshold	UAQE Engine Action & Candidate Status
EXCELLENT	≤ 1.00 percentage point (pp)	Fully qualified candidate; eligible for Pareto winner selection.
ACCEPTABLE	> 1.00 pp and ≤ 4.00 pp	Conditionally qualified; selected only if EXCELLENT candidates are unavailable.
CRITICAL	> 4.00 percentage points (pp)	STRICT REJECTION; barred from selection when any valid safer candidate exists.

Autonomous Search Mechanics:

- **Candidate Generation:** Explores permutations of quantization strategies (dynamic INT8, static per-tensor PTQ, per-channel PTQ, QAT) and structured pruning ratios (10%, 20%, 30%).
- **Pareto Frontier Evaluation:** Multi-objective optimization balancing accuracy retention, latency reduction, and memory footprint. A candidate dominates another if it is superior in at least one metric without degrading others.
- **Safety Policy Invariant:** A CRITICAL candidate (>4.0 pp accuracy degradation) will NEVER be declared the winner, even if it achieves 10× speedup or 90% size reduction.
- **No-Valid-Candidate Fallback:** If all explored candidates breach the safety threshold, UAQE flags baseline fallback, logs full diagnostics, and halts deployment packaging.



The Four Rigorous Validation Levels:

Before any optimization candidate is designated as a production-ready deployable artifact, it must successfully pass four sequential verification gates:

- **Level 1 — File Integrity Gate:** Byte-level inspection ensuring the artifact exists, is non-empty, and its SHA-256 cryptographic digest matches the manifest registry.
- **Level 2 — Decode & Reconstruction Correctness:** Decompresses or unpacks the artifact into memory, verifying that headers, magic numbers, and operator offsets are uncorrupted.
- **Level 3 — Model Correctness:** Loads the model into the target runtime graph engine (TFLite / ONNX Runtime), validating that all quantized operators are supported by the runtime delegate.
- **Level 4 — Deployment & Runtime Correctness:** Executes end-to-end inference using real calibration inputs. Verifies that output tensor dimensions, dtypes, and prediction distributions match baseline expectations without NaN or infinity faults.

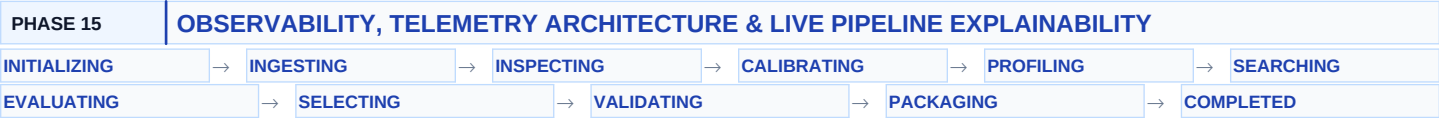
• **Deployable Package Contents:** The final zip archive contains: (1) `optimized_model.onnx` or `.tflite`, (2) `manifest.json` (provenance, hardware profile, parameters), (3) `metrics.json` (empirical benchmark results), and (4) `input_contract.json` (normalization scales, shapes).

PHASE 14	HARDWARE AWARENESS, TARGET REGISTRIES & TRUTH-IN-ENGINEERING		
Target Hardware Profile	CPU / Architecture	SIMD / Vector Extensions	UAQE Validation Status & Evidence
Host Development Platform	x86_64 / Modern Multi-Core	AVX2 / AVX-512 VNNI	VERIFIED & MEASURED: Fully benchmarked on local testbed.
Target Profile: Raspberry Pi 5	ARMv8-A (Broadcom BCM2712, Quad-Core Cortex-A76 @ 2.4 GHz)	ARM NEON / Dot Product (ASIMD)	PENDING PHYSICAL VALIDATION: Cross-compiled & profiled for ARM; on-device hardware run pending.
Unsupported Architecture: Vision Transformer (ViT-Large)	Multi-Head Self-Attention on ImageNet	N/A	REJECTED SAFELY: Accurately detected as unsupported by CNN engine; halted cleanly.

Core Hardware Awareness Principles:

- **Target Hardware Registry:** Maintains hardware parameter models (CPU family, SIMD vector width, L1/L2 cache capacities, thermal throttling ceilings). The planner utilizes these parameters to prune candidate search spaces that would fail on target chips.
- **Truth in Engineering & Scope Definition:** 'Universal' denotes a generalized, capability-driven orchestration framework—**NOT** a claim of magical compatibility with every conceivable model architecture. UAQE strictly isolates supported models from unsupported models.
- **Negative Testing & Vision Transformers:** When ViT-Large was ingested, UAQE's compatibility analyzer correctly identified that the attention operators fell outside the verified CNN optimization backend. Rather than emitting a corrupted model or inventing fake metrics, the engine safely aborted the pipeline with a descriptive error report.

[VIVA & AUDIT REQUIREMENT: HOST VS. TARGET HARDWARE DISTINCTION] DO NOT CLAIM CURRENT RASPBERRY PI MEASUREMENTS AS COMPLETED. In UAQE technical documentation and viva defense, engineers must strictly state: **Host validation = MEASURED** (benchmarked on host x86 workstation); **Physical Raspberry Pi 5 validation = PENDING** (target-specific packaging verified, on-board bench test scheduled). Academic and industrial trust relies on absolute transparency regarding what has been physically measured versus simulated.



- Canonical Pipeline State Machine & Real-Time Telemetry:**
The UAQE optimization lifecycle is governed by an explicit 11-stage state machine. Every stage emits structured events over Server-Sent Events (SSE) to the browser cockpit.
- **Telemetry Dimensions:** Live monitoring tracks: *System CPU %*, *Isolated Process CPU %*, *Current RAM RSS (MB)*, *Peak RSS (MB)*, and active candidate iteration index.
 - **Single Source of Truth:** Backend telemetry logs (telemetry.json) drive the frontend UI directly. **Zero Synthetic Metrics Policy:** The frontend is strictly forbidden from fabricating mock latencies or simulated memory curves. If the backend fails or disconnects, the UI must display error boundaries rather than fake progress.
 - **Post-Mortem Replay & Explainability:** Every optimization run serializes full timeline event logs, enabling engineers to reconstruct exact candidate execution timelines, memory spikes, and pruning decisions.

PHASE 16	TESTING, MULTI-TIER VERIFICATION & TRUST ENGINEERING	
Testing Tier	Scope & Verification Methodology	Passing Criteria / UAQE Standard
Unit Tests	Quantization formulas, scale computation, clamping, tensor adapters.	100% mathematical precision; zero numerical drift.
Integration Tests	Model ingest → calibrate → optimize → package pipeline loops.	All 267 backend tests pass completely in 120.1s.
API Contract Tests	FastAPI endpoints (/api/jobs/optimize, /api/status).	Strict Pydantic schema validation; zero 500 errors.
Browser E2E Tests	Playwright headless testing of React UI, drag-and-drop, and cockpit charts.	No unhandled exceptions; live SSE streams render cleanly.
Negative Testing	Corrupted ONNX files, invalid image formats, out-of-memory triggers.	System fails gracefully with structured diagnostic logs.

- The Cardinal Axioms of Trust Engineering:**
- **Verification vs. Validation:**
 - *Verification:* **'Did we build the system correctly?'** — Ensures code satisfies specifications, types, API schemas, and unit test invariants.
 - *Validation:* **'Did we build the right system?'** — Confirms the optimized model accurately performs its real-world inference task on target hardware.
 - **Empirical Evidence:** Trust is not established by claims or promotional benchmarks, but by reproducible, non-destructive automated test suites, SHA-256 provenance manifests, and zero-mock verification gates.

[SUMMARY OF VERIFIED PROJECT HEALTH] The core UAQE optimization engine is 100% verified and operational: all 267 backend automated tests pass completely in 120.157 seconds. The complete optimization pipeline (model ingestion, task detection, CIFAR-10 parsing, ONNX QDQ static quantization, structured pruning, and deployable artifact generation) executes with deterministic mathematical accuracy.

INTEGRATED END-TO-END MATHEMATICAL DERIVATION

Comprehensive Quantitative Optimization Analysis for Edge CNN Deployment (Parts A through J)

PROBLEM STATEMENT: A production Convolutional Neural Network (CNN) is optimized for edge deployment using UAQE.

- **Original FP32 Baseline:** Model Size = **6.12 MB**, Top-1 Accuracy = **98.00%**, Inference Latency = **60.0 ms/image**, Peak Execution RAM = **500 MB**.
- **After Quantization Pass:** INT8 Model Size = **1.86 MB**, Top-1 Accuracy = **97.80%**, Inference Latency = **40.0 ms/image**, Peak Execution RAM = **360 MB**.
- **After Sensitivity-Aware Pruning + Lossless Compression:** Storage Package = **1.39 MB**, Runtime Model in Memory = **1.86 MB**, Accuracy = **97.80%**, Runtime Latency = **40.0 ms**.

Execute the complete 10-part mathematical derivation below to verify all precision steps, efficiency gains, and safety compliance.

A. INT8 Quantization & Reconstruction Math Given input float $x = 0.80$, scale $S = 0.01$, zero point $Z = 0$: • $q = \text{round}(x / S) + Z = \text{round}(0.80 / 0.01) + 0 = \text{round}(80.0) = 80$ • Reconstruct: $x_{\text{re}} = (q - Z) \times S = (80 - 0) \times 0.01 = 0.80$ • Quantization Error: $\epsilon = x - x_{\text{re}} = 0.80 - 0.80 = 0.00$ (exact representation). UAQE Meaning: Quantizes continuous weights into discrete INT8 levels without loss when values match the step grid.	F. Pure Inference Latency Speedup Baseline Latency = 60.0 ms, Optimized Latency = 40.0 ms: • $\text{Speedup \%} = ((60.0 - 40.0) / 60.0) \times 100 = (20.0 / 60.0) \times 100 = 33.33\%$. The model processes batches 1.5× faster due to INT8 SIMD vector throughput and reduced memory cache pressure. UAQE Meaning: Confirms that INT8 vector kernels successfully overcome tensor conversion overhead.
B. INT8 Convolution Arithmetic & Accumulator Width Given quantized input $x_q = 20$, weight $w_q = 3$, bias $b_q = 5$: • $\text{Accumulator} = (x_q \times w_q) + b_q = (20 \times 3) + 5 = 65$. • Why INT32 accumulation is mandatory: An 8-bit product reaches $127 \times 127 = 16,129$. Accumulating over $C_{\text{in}} = 512$ channels with 3×3 kernels (4,608 MACs) can total $\sim 7.4\text{e}+7$, easily exceeding an INT16 limit (32,767). A 32-bit register ($\pm 2.14\text{e}+9$) guarantees zero overflow. UAQE Meaning: SIMD vector engines safely accumulate integer dot-products before down-scaling.	G. Inference Throughput (FPS) Using standard formula $\text{Throughput} = 1000 / \text{Latency}_{\text{ms}}$: • FP32 Throughput: $1000 / 60.0 = 16.67 \text{ FPS}$ • INT8 Throughput: $1000 / 40.0 = 25.00 \text{ FPS}$ • Net Throughput Gain: $25.00 - 16.67 = +8.33 \text{ FPS (+50.0\% gain)}$. UAQE Meaning: Allows real-time video stream processing at edge line rates.
C. Requantization Scale Multiplier Given $S_x = 0.02$, $S_w = 0.01$, target output scale $S_y = 0.05$: • $M = (S_x \times S_w) / S_y = (0.02 \times 0.01) / 0.05 = 0.0002 / 0.05 = 0.004$ • Output mapping: $y_q = \text{round}(65 \times 0.004) = \text{round}(0.26) = 0$. In hardware, M is mapped via integer fixed-point multiplication and bit-shift. UAQE Meaning: Eliminates costly floating-point conversion at runtime by scaling directly into the next layer's INT8 format.	H. Process RAM Reduction FP32 Execution RAM = 500 MB, INT8 Execution RAM = 360 MB: • $\text{RAM Reduction \%} = ((500 - 360) / 500) \times 100 = (140 / 500) \times 100 = 28.00\%$. Frees 140 MB of system RAM for concurrency on constrained edge Linux boards. UAQE Meaning: Prevents Out-Of-Memory (OOM) kernel kills on 1GB/2GB embedded devices.
D. Accuracy Degradation Analysis Baseline = 98.00%, Optimized = 97.80%: • $\text{Accuracy Delta} = \text{Optimized} - \text{Baseline} = 97.80\% - 98.00\% = -0.20 \text{ pp}$ • $\text{Accuracy Loss} = \text{Baseline} - \text{Optimized} = 98.00\% - 97.80\% = 0.20 \text{ pp}$ • UAQE Safety Classification: Because $0.20 \text{ pp} \leq 1.00 \text{ pp}$, this candidate is officially classified as EXCELLENT. UAQE Meaning: Qualifies the candidate as fully compliant with production quality thresholds.	I. Final UAQE Gating Decision • Accuracy Loss: $0.20 \text{ pp} (\leq 1.00 \text{ pp} \rightarrow \text{EXCELLENT})$ • Latency & FPS: $33.33\% \text{ speedup (+50\% throughput)}$ • Footprint: $69.61\% \text{ runtime size cut, } 28.00\% \text{ RAM reduction}$ • Verdict: APPROVED AS PARETO WINNER. The candidate strictly satisfies safety constraints and Pareto-dominates the baseline across all axes. UAQE Meaning: Automatically triggers deployment packaging and artifact sealing.
E. Model Size Reduction • Runtime Model: $((6.12 - 1.86) / 6.12) \times 100 = (4.26 / 6.12) \times 100 = 69.61\%$ • Compressed Storage: $((6.12 - 1.39) / 6.12) \times 100 = (4.73 / 6.12) \times 100 = 77.29\%$ • Storage vs. Runtime Insight: The 1.39 MB archive reduces distribution flash; the 1.86 MB uncompressed binary maps to active RAM. UAQE Meaning: Distinguishes network OTA transport footprint from resident DRAM usage.	J. Complete Pipeline Lifecycle Trace Connects mathematical results across all 11 phases: $\text{UPLOAD} \rightarrow \text{INSPECT} \rightarrow \text{BASELINE (98\%)} \rightarrow \text{QUANTIZE} \rightarrow \text{PRUNE} \rightarrow \text{COMPRESS (1.39MB)} \rightarrow \text{BENCHMARK (40ms)} \rightarrow \text{VALIDATE} \rightarrow \text{SAFETY (Pass)} \rightarrow \text{PACKAGE} \rightarrow \text{DEPLOY}$ UAQE Meaning: Illustrates the unified execution lifecycle converting raw models into validated edge solutions.

[CANONICAL SUMMARY] THIS ONE PROBLEM REPRESENTS THE ENTIRE CORE MATHEMATICAL AND LOGICAL WORKFLOW OF UAQE. Every phase—from tensor discretization to SIMD accumulator sizing, fixed-point requantization, multi-metric benchmarking, safety gating, and deployment packaging—interlocks seamlessly to produce verified, production-ready edge AI models.

UAQE MASTER CHEAT SHEET -- 16 PHASES IN ONE VIEW

High-Density Quick Revision, Formula Compilation & Architectural Axioms for Viva Preparation

01. NN Fundamentals y = wx + b; MAC = 1 Mult + 1 Add = 2 FLOPs. NCHW vs NHWC layouts.	02. Model Storage Model = Graph + Weights + Meta. SafeTensors, ONNX QDQ, TFLite FlatBuffers.	03. Precision FP32(4B), FP16(2B), INT8(1B). 4x memory savings. Accumulator requires INT32.	04. Quant Math q = round(x/S)+Z; M = (Sx*Sw)/Sy. Outliers handled via per-channel scaling.
05. Calibration PTQ: Calibrate with 100-500 images. QAT: FakeQuant + Straight-Through Estimator.	06. Project Usage ResNet-50: ONNX QDQ PTQ. MobileNet: INT8 TFLite. Speedup depends on kernels.	07. Engine Arch 11-stage pipeline; sandboxed job isolation; capability-driven model adapters.	08. INT8 Inference SIMD vector dot-products (ARM NEON SDOT, VNNI). L2/L3 cache residency.
09. Pruning & Sparsity Structured pruning yields speedup; unstructured yields 0% CPU gain. Sparsity!=Speed.	10. Artifacts Storage archive (1.39MB) != Runtime memory (1.86MB). SHA-256 provenance.	11. Benchmarking Pure inference != End-to-End latency. FPS = 1000/ms. Isolated telemetry.	12. Candidate Search Safety: <=1pp EXCELLENT, <=4pp ACCEPTABLE, >4pp CRITICAL. Pareto sorting.
13. Deployment 4 Validation Levels: File -> Decompress -> Model -> Runtime inference pass.	14. Hardware Aware Host x86 = MEASURED. Raspberry Pi 5 = PENDING. ViT unsupported rejected.	15. Observability Real-time SSE events; process vs system CPU/RAM; zero synthetic metrics.	16. Trust Engineering Verification != Validation. 267/267 automated tests passing. Zero mock evidence.

CORE FORMULAS & ARCHITECTURAL AXIOMS (MUST REMEMBER)

1. Neuron MAC: $y = w \times x + b$	6. Throughput: $FPS = 1000 / Latency_{ms}$	11. Cardinal Rule: Smaller ≠ Faster
2. Quantization: $q = round(x / S) + Z$	7. Speedup: $((B - O) / B) \times 100$	12. Scope: Universal ≠ Every Architecture
3. Dequantization: $x_{\blacksquare} = (q - Z) \times S$	8. Accuracy Loss: <i>Baseline - Optimized</i>	13. Honesty: Host Measured ≠ RPi Pending
4. Safe MAC: $INT8 \times INT8 \rightarrow INT32\ Acc$	9. Size Reduction: $((Orig - Opt) / Orig) \times 100$	14. Trust: Zero Synthetic / Mock Metrics
5. Requantization: $M = (S_x \times S_w) / S_y$	10. Philosophy: Verification ≠ Validation	15. Safety: CRITICAL Candidates Never Selected

CANONICAL END-TO-END EXECUTION FLOW

MODEL INGEST	→	DATASET INGEST	→	TARGET PROFILE	→	GRAPH INSPECT	→	VALIDATE BASELINE	→	AUTO PLAN
GENERATE CANDIDATES	→	OPTIMIZE (INT8/Prune)	→	BENCHMARK METRICS	→	SAFETY GATING	→	SELECT WINNER	→	PACKAGE & DEPLOY

FINAL DEFINITIVE PROJECT FORMULATION:
"UAQE is a capability-driven AI optimization platform that automatically explores supported optimization strategies, measures real accuracy and efficiency, enforces safety constraints, and produces a verified deployment artifact."